

Computational Physics

Lecture 3: Numerical Integration

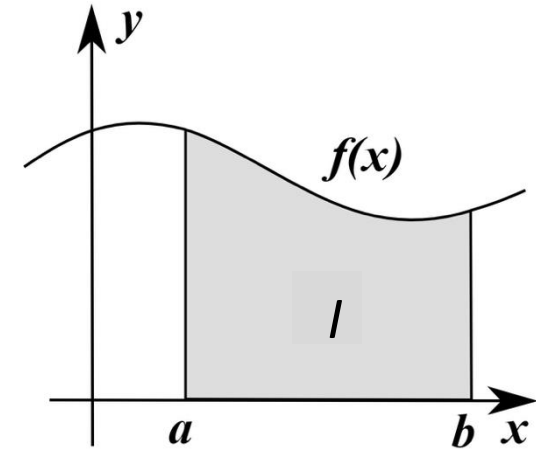
- Basic methods for numerical integration (trapezoid, Simpson)
- Gaussian quadrature
- Integrals over infinite ranges

References: *Computational Physics* by Mark Newman, Chapter 5
<https://websites.umich.edu/~mejn/cp2/chapters/chapter5.pdf>
Numerical Recipes Third Edition by W.H. Press et al.

Numerical integration

Generic problem: evaluate the following integral of a function $f(x)$ in a specified domain from $x=a$ to $x=b$

$$I(a, b) = \int_a^b f(x) dx. \quad \text{equivalent to computing the area:}$$

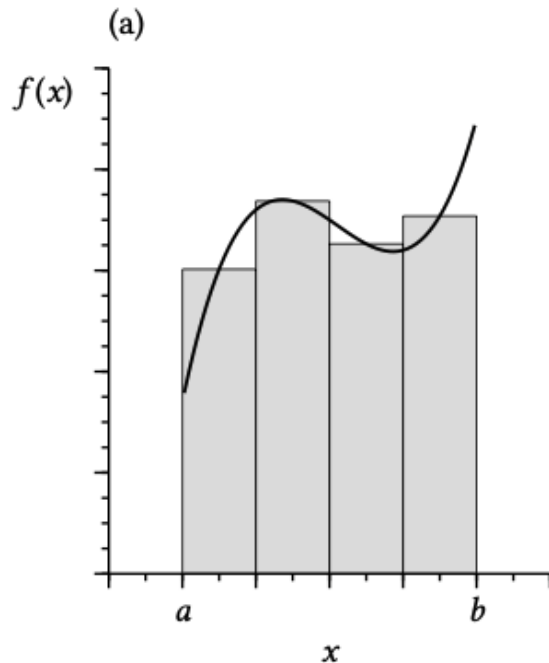


We need numerical integration when

- Cannot/difficult to integrate analytically
- Only know the integrand $f(x)$ at certain points

Rectangle rule

Easiest approach: approximate the area in the following way

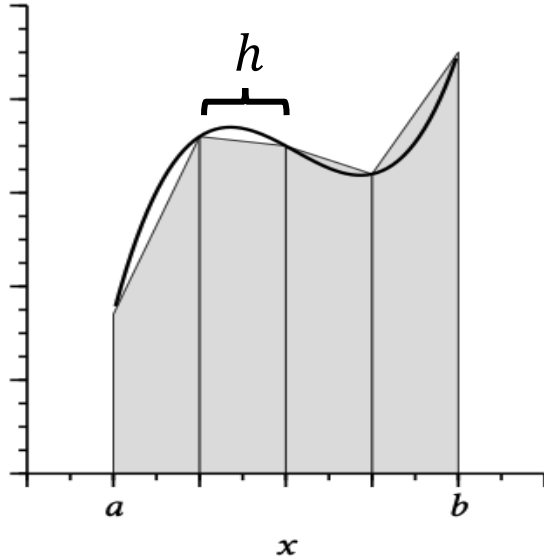


- 1) Sample the function at certain equidistant points
- 2) Consider rectangles with a width equal to the distance between two neighbor points and height equal to one value of function taken in that interval (can be e.g. the first value or the middle one)
- 3) Compute the rectangular areas and sum them up to obtain an approximated value of the area

The area however is not very close to the one of the curve

Trapezoidal rule

A better approach is to use trapezoids (**trapezoidal rule**)



1) Divide $[a, b]$ in N slices, each slice has a width $h = (b-a)/N$, so that each slice has a domain $[a+(k-1)h, a+kh]$ with $k=1, \dots, N$

2) The trapezoid in each slice is identified by the function's values at the border domain $f(a+(k-1)h)$ and $f(a+kh)$ and has an area

$$A_k = \frac{1}{2}h[f(a+(k-1)h) + f(a+kh)]$$

3) The total area is $I(a, b) \simeq \sum_{k=1}^N A_k = \frac{1}{2}h \sum_{k=1}^N [f(a+(k-1)h) + f(a+kh)] = h[\frac{1}{2}f(a) + f(a+h) + f(a+2h) + \dots + \frac{1}{2}f(b)]$

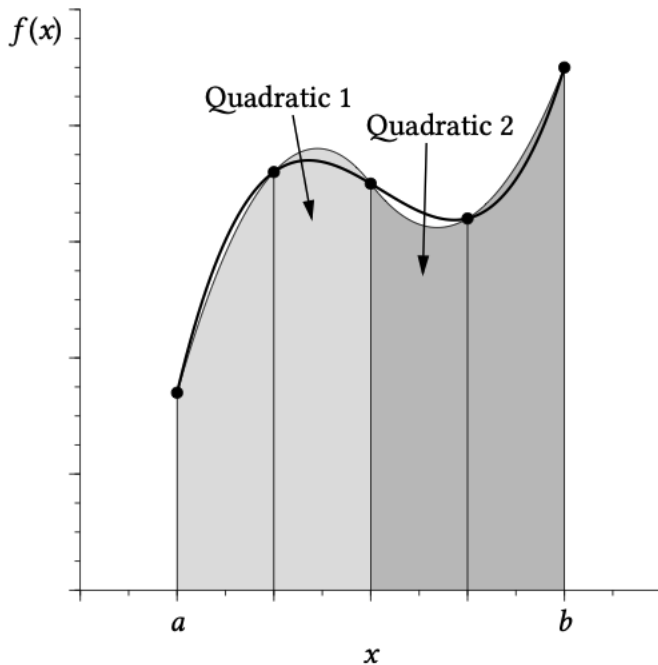
$$I(a, b) \simeq h \left[\frac{1}{2}f(a) + \frac{1}{2}f(b) + \sum_{k=1}^{N-1} f(a+kh) \right]$$

Larger N means smaller error
(to be discussed later)

Simpson's rule

The trapezoidal rule is often perfectly adequate for calculations where no great accuracy is required. But we can do a better job sometimes approximating the function with curves of some kind instead of straight lines.

For **Simpson's rule** we use quadratic curves:



- 1) take now three uniformly spaced points (or equivalently two adjacent slices)
- 2) fit a quadratic curve through the three points
- 3) compute the area under the quadratic curve
- 4) sum all estimated pair slice areas

Simpson's rule

Start fitting a quadratic curve under 3 points and compute the area

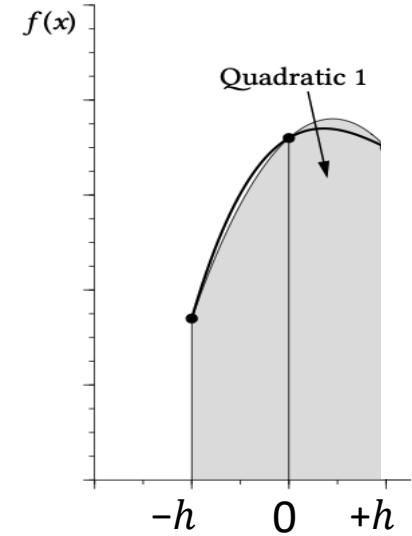
Consider $f(x)$, take three points at $x = -h, 0$, and $+h$ and fit a quadratic $Ax^2 + Bx + C$ so find A, B, C such that $f(x)$ at these three points correspond the quadratic function:

$$f(-h) = Ah^2 - Bh + C, \quad f(0) = C, \quad f(h) = Ah^2 + Bh + C.$$

Invert the equations and obtain A, B, C

$$A = \frac{1}{h^2} \left[\frac{1}{2}f(-h) - f(0) + \frac{1}{2}f(h) \right], \quad B = \frac{1}{2h} [f(h) - f(-h)], \quad C = f(0),$$

The area under the quadratic curve is $\int_{-h}^h (Ax^2 + Bx + C) dx = \frac{2}{3}Ah^3 + 2Ch = \frac{1}{3}h[f(-h) + 4f(0) + f(h)]$.



So we just need the values of $f(x)$ at the three evenly spaced points and input them in the formula. These can be any three points, not just $-h, 0, h$

Simpson's rule

Sum up the area of each successive double slice and obtain the total area (need even number of slices)

$$\begin{aligned} I(a, b) &\simeq \frac{1}{3}h[f(a) + 4f(a+h) + f(a+2h)] + \frac{1}{3}h[f(a+2h) + 4f(a+3h) + f(a+4h)] + \dots + \frac{1}{3}h[f(a+(N-2)h) + 4f(a+(N-1)h) + f(b)] = \\ &= \frac{1}{3}h[f(a) + 4f(a+h) + 2f(a+2h) + 4f(a+3h) + \dots + f(b)] \end{aligned}$$

$$I(a, b) \simeq \frac{1}{3}h \left[f(a) + f(b) + 4 \sum_{\substack{k \text{ odd} \\ 1 \dots N-1}} f(a+kh) + 2 \sum_{\substack{k \text{ even} \\ 2 \dots N-2}} f(a+kh) \right]$$

For summation in Python, you can use directly “for k in range(1,N,2)” for the odd terms or “for k in range(2,N,2)” for even ones, or alternatively a single summation from 1 to N/2:

$$I(a, b) \simeq \frac{1}{3}h \left[f(a) + f(b) + 4 \sum_{k=1}^{N/2} f(a + (2k-1)h) + 2 \sum_{k=1}^{N/2-1} f(a + 2kh) \right]$$

Example

Integrate $x^4 - 2x + 1$ from $x=0$ to $x=2$ using $N=10$, 100 using the trapezoidal and Simpson's rules and compare to each other and the analytical value

```
#Trapezoidal rule
def f(x):
    return x**4 - 2*x + 1
N = 10
a = 0.0
b = 2.0
h = (b-a)/N
s = 0.5*f(a) + 0.5*f(b)
for k in range(1,N):
    s += f(a+k*h)
print(h*s)
```

```
#Simpson's rule
def f(x):
    return x**4 - 2*x + 1
N = 10          # need to be even
a = 0.0
b = 2.0
h = (b - a) / N
s = f(a) + f(b)
for k in range(1, N):
    xk = a + k * h
    if k % 2 == 0:
        s += 2 * f(xk)
    else:
        s += 4 * f(xk)
print((h / 3) * s)
```

$$\int_0^2 (x^4 - 2x + 1) dx = \left[\frac{1}{5}x^5 - x^2 + x \right]_0^2 = 4.4.$$

For $N=10$ we have $I=4.50656$ and $I=4.400426666$ for the trapezoidal and Simpson's method, to compare with 4.4, so an error of 2% and 0.0096%, while for $N=100$ $I=4.40000004266$ and $I= 4.400000042666667$

Simpson's method reduces the error substantially, at the cost of only a few extra calculations.

Euler–Maclaurin error formula for trapezoidal rule

Aside from the rounding error, we have here a major error contribution, the **approximation error** as we are approximating area.

For the trapezoidal rule,
Euler-Maclaurin rule

$$\int_a^b f(x) \, dx = \underbrace{\frac{1}{2}h \sum_{k=1}^N [f(x_{k-1}) + f(x_k)]}_{\text{estimated integral value with this method}} + \underbrace{\frac{1}{12}h^2 [f'(a) - f'(b)]}_{\text{first Euler–Maclaurin correction term}} + O(h^4).$$

$$\delta = \frac{1}{12}h^2 [f'(a) - f'(b)]$$

δ is an estimation of the error in computing the integral. Note that the terms proportional to h^3 cancel out.

The trapezoidal rule is a first-order integration rule: it is accurate up to and including terms proportional to h and the leading-order approximation error is of order h^2 (**accurate to** $O(h)$ and has an **error** $O(h^2)$)

So one improves the result decreasing h . However, it can be decreased up to machine precision, where the rounding error starts to count

Euler–Maclaurin error formula for trapezoidal rule

Let's derive the error formula:

- divide interval $[a,b]$ in sample points $x_k = a + kh$
- Taylor expansion of $f(x)$ around x_{k-1}
$$f(x) = f(x_{k-1}) + (x - x_{k-1})f'(x_{k-1}) + \frac{1}{2}(x - x_{k-1})^2 f''(x_{k-1}) + \dots$$
- Integrate from x_{k-1} to x_k
$$\int_{x_{k-1}}^{x_k} f(x) \, dx = f(x_{k-1}) \int_{x_{k-1}}^{x_k} dx + f'(x_{k-1}) \int_{x_{k-1}}^{x_k} (x - x_{k-1}) \, dx + \frac{1}{2}f''(x_{k-1}) \int_{x_{k-1}}^{x_k} (x - x_{k-1})^2 \, dx + \dots$$
- Substitute $u = x - x_{k-1}$ and perform integral

$$\begin{aligned} \int_{x_{k-1}}^{x_k} f(x) \, dx &= f(x_{k-1}) \int_0^h du + f'(x_{k-1}) \int_0^h u \, du + \frac{1}{2}f''(x_{k-1}) \int_0^h u^2 \, du + \dots \\ &= hf(x_{k-1}) + \frac{1}{2}h^2 f'(x_{k-1}) + \frac{1}{6}h^3 f''(x_{k-1}) + O(h^4), \end{aligned}$$

- Perform same operation expanding around x_k and integrating from x_{k-1} to x_k

$$\int_{x_{k-1}}^{x_k} f(x) \, dx = hf(x_k) - \frac{1}{2}h^2 f'(x_k) + \frac{1}{6}h^3 f''(x_k) - O(h^4).$$

Euler–Maclaurin error formula for trapezoidal rule

- Average the two results

$$\int_{x_{k-1}}^{x_k} f(x) \, dx = \frac{1}{2}h[f(x_{k-1}) + f(x_k)] + \frac{1}{4}h^2[f'(x_{k-1}) - f'(x_k)] + \frac{1}{12}h^3[f''(x_{k-1}) + f''(x_k)] + O(h^4).$$

- Sum over all slices

$$\int_a^b f(x) \, dx = \sum_{k=1}^N \int_{x_{k-1}}^{x_k} f(x) \, dx = \boxed{\frac{1}{2}h \sum_{k=1}^N [f(x_{k-1}) + f(x_k)]} + \frac{1}{4}h^2[f'(a) - f'(b)] + \boxed{\frac{1}{12}h^3 \sum_{k=1}^N [f''(x_{k-1}) + f''(x_k)]} + O(h^4).$$

trapezoidal rule

trapezoidal rule applied to the function $f''(x)$

$$\int_a^b f''(x) \, dx = \frac{1}{2}h \sum_{k=1}^N [f''(x_{k-1}) + f''(x_k)] + O(h^2) \xrightarrow{\text{rearranging}} \frac{1}{12}h^3 \sum_{k=1}^N [f''(x_{k-1}) + f''(x_k)] = \frac{1}{6}h^2 \int_a^b f''(x) \, dx + O(h^4) = \frac{1}{6}h^2[f'(b) - f'(a)] + O(h^4),$$

This formula can be substituted back inside the integral

- We arrive at the Euler-MacLaurin formula

$$\int_a^b f(x) \, dx = \frac{1}{2}h \sum_{k=1}^N [f(x_{k-1}) + f(x_k)] + \frac{1}{12}h^2[f'(a) - f'(b)] + O(h^4).$$

The h^3 term cancels out, and one finds in general that only even powers of h survive

Error formula for Simpson's rule

For the Simpson's rule, the error would be

$$\delta = \frac{1}{180}h^4[f'''(a) - f'''(b)]$$

The error is $O(h^4)$, so two orders better than the trapezoidal rule.

As a rule of thumb, for Simpson's rule you don't need more than $N=10^4$ slides, limit in which you reach rounding error.

For the trapezoidal rule it would be $N=10^8$

Using one method over the other really depends on the integral: it is not always guaranteed that one method do better than the other, as we have a dependencies in δ over the derivatives values as well.

Practical estimation of error

What if we don't have access to $f(x)$ derivatives formula? Let's see a **practical method for trapezoid rule**.

Integrate using number of steps N_1 , and then double it $N_2 = 2N_1$, or equivalently halve the step:

$$h_1 = \frac{b-a}{N_1} \longrightarrow h_2 = \frac{h_1}{2}$$

Because the initial error scales as h_1^2 we effectively reduced the error to a quarter $h_2^2 = h_1^2/4$

$$\text{error} \propto h_1^2 \longrightarrow \propto \frac{h_1^2}{4}$$

And the real value I of a integral can be expressed as

$$I = I_1 + ch_1^2, \longrightarrow I = I_2 + ch_2^2.$$

I_1, I_2 : integral estimates with the two steps
 c : common constant dependent only on f and not h (see MacLaurin formula)

Equating the two expressions:

$$I_1 + ch_1^2 = I_2 + ch_2^2, \longrightarrow I_2 - I_1 = ch_1^2 - ch_2^2 = 3ch_2^2, \longrightarrow \delta_2 = ch_2^2 = \frac{1}{3}(I_2 - I_1)$$

integral error estimate

Numerical integration: practical integrals error estimation

δ_2 is a simple way to estimate the error without using the Euler–Maclaurin formula

Rather than the value with sign take the absolute value

For trapezoidal rule

$$\delta_2 = \frac{1}{3} |I_2 - I_1|$$

For Simpson's rule

$$\delta_2 = \frac{1}{15} |I_2 - I_1|$$

Numerical integration: integrals error exercise

Take now the previous integral of $x^4 - 2x + 1$, estimate the error using the Euler–Maclaurin and the heuristic formulas we saw and compare with the error obtained directly using the correct integral

```
#Trapezoidal rule with error estimates
def f(x):
    return x**4 - 2*x + 1
def fp(x):
    return 4*x**3 - 2          # f'(x) for Euler–Maclaurin

def trapezoid(N, a, b):
    h = (b - a) / N
    s = 0.5*f(a) + 0.5*f(b)
    for k in range(1, N):
        s += f(a + k*h)
    return h*s, h

a, b = 0.0, 2.0
N = 10                      # fine grid (must be even if you also use N/2)

I2, h2 = trapezoid(N, a, b)  # I2 = T(h)
I1, h1 = trapezoid(N//2, a, b) # I1 = T(2h)

# Heuristic leading error of I2:  $\delta_2 = c h^2 = (I_2 - I_1)/3$ 
delta2 = abs((I2 - I1) / 3.0)

# Euler–Maclaurin first correction for I2:  $(h^2/12)[f'(a) - f'(b)]$ 
em2 = abs((h2**2 / 12.0) * (fp(a) - fp(b)))

exactI = 4.4
print("T(N) = I2:", I2)
print("|I2-I|:", I2-exactI)
print("heuristic  $\delta_2 = (I_2-I_1)/3$ :", delta2)
print("EM correction on I2:", em2)

T(N) = I2: 4.50656
|I2-I|: 0.10655999999999999
heuristic  $\delta_2 = (I_2-I_1)/3$ : 0.10613333333333334
EM correction on I2: 0.10666666666666669
```

```
# Simpson's rule with error estimates
def f(x):
    return x**4 - 2*x + 1

def f4(x):
    return 24.0              # f^{(4)}(x), constant for this f

def simpson(N, a, b):
    if N % 2 != 0:
        raise ValueError("N must be even")
    h = (b - a) / N
    s = f(a) + f(b)
    for k in range(1, N):
        xk = a + k*h
        s += 4*f(xk) if k % 2 else 2*f(xk)
    return (h/3.0)*s, h

a, b = 0.0, 2.0
N = 12                      # multiple of 4 so that N/2 is even

I2, h2 = simpson(N, a, b)    # I2 = S(h)
I1, h1 = simpson(N//2, a, b) # I1 = S(2h)

# Heuristic leading error of I2:  $\delta_2 = c h^4 = (I_2 - I_1)/15$ 
delta2 = abs((I2 - I1) / 15.0)

# Euler–Maclaurin / Simpson remainder:  $-(b-a) h^4 / 180 * f^{(4)}$ 
em2 = abs(-(b - a) * h2**4 / 180.0 * f4(0.0))

exactI = 4.4
print("S(N) = I2:", I2)
print("|I2-I|:", abs(I2 - exactI))
print("heuristic  $\delta_2 = (I_2-I_1)/15$ :", delta2)
print("EM/Simpson correction on I2:", em2)

S(N) = I2: 4.4002057613168715
|I2-I|: 0.0002057613168711143
heuristic  $\delta_2 = (I_2-I_1)/15$ : 0.0002057613168724141
EM/Simpson correction on I2: 0.00020576131687242795
```

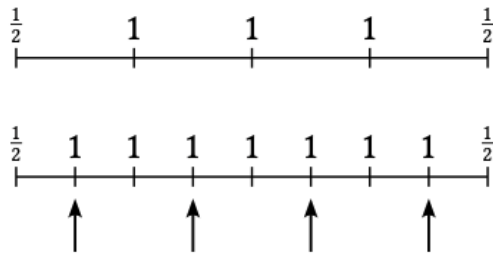
Choice of N : Adaptive integration

Suppose we want an accuracy of four decimal. In order e.g. to not waste computational resources, we want an N that gives this exact precision

Adaptive integration method:

- 1) Start estimating I with a small value of N
- 2) Double repeatedly N and estimate I and error.
- 3) Stop when error is smaller than target accuracy

For the trapezoidal rule, you also don't need to compute all $f(x_k)$ values for each N :



when doubling N , original points are still present. Need to add new ones with correct multipliers:

$$I_i = \frac{1}{2}I_{i-1} + h_i \sum_{\substack{k \text{ odd} \\ 1 \dots N_i-1}} f(a + kh_i)$$

In the book you will see the equivalent procedure for Simpson's rule